Soft Actor-Critic (SAC)

Samsung AI Expert

Jul 11, 2025

Giho Kim

SAC - Review

SAC - Review

How to incentivize exploration?

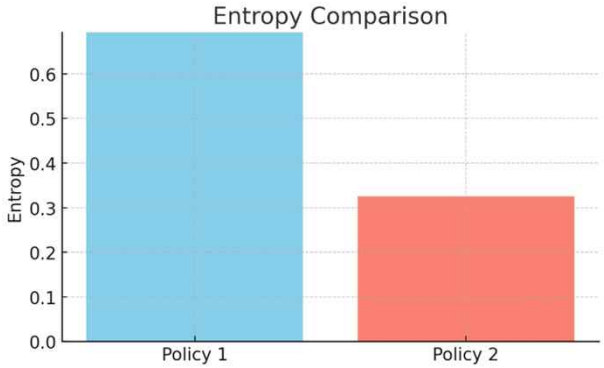
idea : augment reward as follows:

$$\sum_{t=0}^{T-1} \left(r(s_t, a_t) + \alpha \mathscr{H}(\pi(\cdot|s_t)) \right)$$

where $\mathscr{H}(\pi(\cdot|s_t))$: **entropy** of action distribution $\pi(\cdot|s_t)$

SAC - Review

What is entropy of policy?



$$H(\pi(\cdot|s)) = -\mathbb{E}_{a\sim\pi(\cdot|s)}[\log \pi(a|s)] = -\int_a \log \pi(a|s)\pi(a|s)da$$

Learning and Decision Systems

SAC - Review : Soft policy evaluation

- Modified Bellman operator:

$$T^{\pi}Q(s,a) := r(s,a) + \gamma \mathbb{E}_{s'}[V(s')],$$

where $V(s) := \mathbb{E}_{a\sim\pi(\cdot|s)}\big[Q(s,a) \underbrace{-\log \pi(a|s)}_{\text{takes into account entropy}}\big]$

SAC - Review : Soft policy improvement

- If no constraint on policies,

$$\pi_{new}(a|s) := \exp\left(\frac{1}{\alpha}[Q^{\pi_{old}}(s,a) - V^{\pi_{old}}(s)]\right)$$

- When constraints need to be satisfied (i.e., $\pi \in \Pi$),
  Information projection:

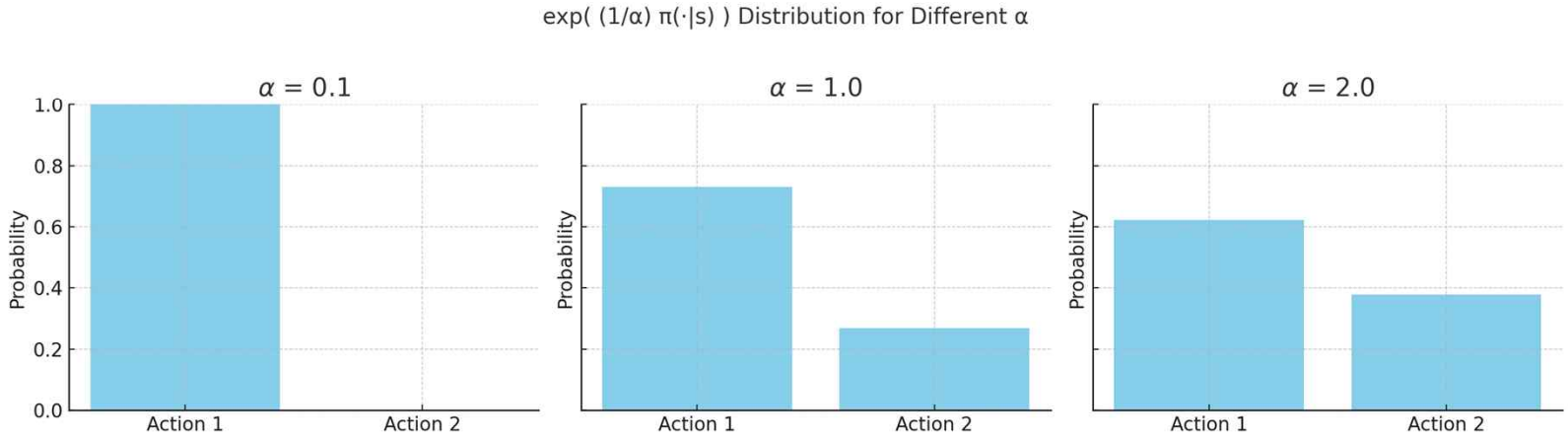
$$\pi_{new}(\cdot|s) \in \underset{\pi'(\cdot|s)\in\Pi}{\arg\min}\, D_{KL}\Bigg(\pi'(\cdot|s) \parallel \underbrace{\exp\left(\frac{1}{\alpha}[Q^{\pi_{old}}(s,\cdot) - V^{\pi_{old}}(s)]\right)}_{\text{target policy}}\Bigg)$$

Learning and Decision Systems

SAC - Review : Soft policy improvement

- If no constraint on policies,

$$\pi_{new}(a|s) := \exp\left(\frac{1}{\alpha}[Q^{\pi_{old}}(s,a) - V^{\pi_{old}}(s)]\right)$$



Learning and Decision Systems

SAC - Review : Convergence to optimal policy

Theorem 1 (Soft Policy Iteration). *Repeated application of soft policy evaluation and soft policy improvement from any* $\pi \in \Pi$ *converges to a policy* π^* *such that* $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ *for all* $\pi \in \Pi$ *and* $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, *assuming* $|\mathcal{A}| < \infty$.

SAC - Implementation

SAC -Implementation

Critic loss

Gaussian actor network $a \sim \mathcal{N}(\mu_\phi(s), \sigma_\phi(s))$

two critic networks $Q_1(s, a; \theta_1),\ Q_2(s, a; \theta_2)$

target & loss construction

critic target :

Remind: Entropy $H(X) = -\sum_{x \in X} p(x) \log p(x) = E[-\log p(x)]$

$$y_j = r_j + \gamma \mathbb{E}_{a \sim \pi(\cdot|s_j)} \left(Q(s'_j, a) - \alpha \log \pi(a|s_j) \right)$$

$$\rightarrow y_j = r_j + \gamma \mathbb{E}_{a \sim \pi_{\phi^-}(\cdot|s_j)} \left(\min_{i=1,2} Q_i(s'_j, a; \theta_i^-) - \alpha \log \pi_{\phi^-}(a|s_j) \right)$$

underestimation rather than overestimation[1]

[1]Fujimoto, Scott, Herke Hoof, and David Meger. "Addressing function approximation error in actor-critic methods." *International conference on machine learning*. PMLR, 2018.

LDS
Learning and Decision Systems

SAC -Implementation

Soft actor loss :

$$\min_{\phi} J_\pi(\phi) := \mathbb{E}_{s_t}\Big[\mathbb{E}_{a_t\sim\pi_\phi(\cdot|s_t)}\Big[\underbrace{\alpha \log \pi_\phi(a_t|s_t) - Q_\theta(s_t,a_t)}_{=:\, f_\phi(a)}\Big]\Big]$$

SAC -Implementation

Soft actor loss :

$$\min_{\phi} J_\pi(\phi) := \mathbb{E}_{s_t}\Big[\mathbb{E}_{a_t\sim\pi_\phi(\cdot|s_t)}\Big[\underbrace{\alpha \log \pi_\phi(a_t|s_t) - Q_\theta(s_t,a_t)}_{=:\, f_\phi(a)}\Big]\Big]$$

Gradient estimation :

$$\nabla_\phi \mathbb{E}_{a\sim\pi_\phi(\cdot|s)}[f_\phi(a)] = \nabla_\phi \int f_\phi(a)\,\pi_\phi(a)\,da$$

$$= \int \big(\nabla_\phi f_\phi(a)\big)\,\pi_\phi(a)\,da + \int f_\phi(a)\,\nabla_\phi \pi_\phi(a)\,da$$

$$= \mathbb{E}_{a\sim\pi_\phi}\big[\nabla_\phi f_\phi(a)\big] + \mathbb{E}_{a\sim\pi_\phi}\big[f_\phi(a)\,\nabla_\phi \log \pi_\phi(a)\big]$$

high variance because of sampling noise

SAC -Implementation

Soft actor loss :

$$\min_{\phi} J_\pi(\phi) := \mathbb{E}_{s_t}\Big[\mathbb{E}_{a_t\sim\pi_\phi(\cdot|s_t)}\Big[\underbrace{\alpha \log \pi_\phi(a_t|s_t) - Q_\theta(s_t, a_t)}_{=:\, f_\phi(a)}\Big]\Big]$$

Gradient estimation with **reparameterization trick**:

$$a = g_\phi(\epsilon, s) = \mu_\phi(s) + \sigma_\phi(s) \odot \epsilon \text{ where } \epsilon \sim \mathcal{N}(0, 1)$$

$$\nabla_\phi \, \mathbb{E}_{a\sim\pi_\phi(\cdot|s)}[f_\phi(a)] = \mathbb{E}_{\epsilon\sim p(\epsilon)}\big[\nabla_\phi f_\phi(g_\phi(\epsilon, s)))\big]$$

the gradients flow through only deterministic part, μ_ϕ and σ_ϕ, during backpropagation

SAC -Implementation

actor loss :

$$\alpha \log \pi_\phi(f_\phi(\epsilon_j, s_j)|s_j) - \min_{i=1,2} Q_i(s_j, f_\phi(\epsilon_j, s_j))$$

Remark. π_ϕ : probability density, f_ϕ : actor network

SAC - Implementation

actor definition - 1st step

```
def forward(self, state, eval=False, with_log_prob=False):
    x = F.relu(self.fc1(state))
    x = F.relu(self.fc2(x))

    mu = self.fc3(x)
    log_sigma = self.fc4(x)
    # clip value of log_sigma, as was done in Haarnoja's implementation of SAC:
    # https://github.com/haarnoja/sac.git
    log_sigma = torch.clamp(log_sigma, -20.0, 2.0)
    sigma = torch.exp(log_sigma)
    distribution = Independent(Normal(mu, sigma), 1)
```

what we are doing here : compute **distribution params** $\mu_\phi(s)$ and $\sigma_\phi(s)$

Learning and Decision Systems

SAC - Implementation

actor definition - 2nd step

```
if not eval:
    # use rsample() instead of sample(), as sample() does not allow back-propagation through params
    u = distribution.rsample()
    if with_log_prob:
        log_prob = distribution.log_prob(u)
        log_prob -= 2.0 * torch.sum((np.log(2.0) + 0.5 * np.log(self.ctrl_range) - u - F.softplus(-2.0 * u)), dim=1)

    else:
        log_prob = None
else:
    u = mu
    log_prob = None
# apply tanh so that the resulting action lies in (-1, 1)^D
a = self.ctrl_range * torch.tanh(u)

return a, log_prob
```

Reparameterization trick to ease computation of $\nabla \log \pi(a|s)$

$$\text{softplus}(x) = \log(1 + e^x)$$

$$u \sim \mathcal{N}(\mu_\phi(s), \sigma_\phi(s)) \quad \longrightarrow \quad a = \tanh(u)$$

SAC - Implementation

```
def __init__(self, dimS, dimA, hidden1, hidden2):
    super(DoubleCritic, self).__init__()
    self.fc1 = nn.Linear(dimS + dimA, hidden1)
    self.fc2 = nn.Linear(hidden1, hidden2)
    self.fc3 = nn.Linear(hidden2, 1)

    self.fc4 = nn.Linear(dimS + dimA, hidden1)
    self.fc5 = nn.Linear(hidden1, hidden2)
    self.fc6 = nn.Linear(hidden2, 1)

def forward(self, state, action):
    x = torch.cat([state, action], dim=1)
    x1 = F.relu(self.fc1(x))
    x1 = F.relu(self.fc2(x1))
    x1 = self.fc3(x1)

    x2 = F.relu(self.fc4(x))
    x2 = F.relu(self.fc5(x2))
    x2 = self.fc6(x2)

    return x1, x2
```

this is how we define twin critics!

SAC - Implementation

```
with torch.no_grad():
        # Get action with log(pi(a|s)) (also gradient)
        next_actions, log_probs = agent.pi(next_obs_batch, with_log_prob=True)

        # To calculate TQ, we need Q(s',pi(s'))
        target_q1, target_q2 = agent.target_Q(next_obs_batch, next_actions)

        # To mitigate overestimation! - Idea from TD3
        target_q = torch.min(target_q1, target_q2)

        # TQ^pi = r + gamma [ Q(s',pi(s')) - alpha H(pi(s')) ]
        # Recall : H = sum[ -P(X) * log(P(x)) ] = E [ -log(P(x)) ]
        # Recall : H \approx -log(P(x))
        TQ = rew_batch + agent.gamma * masks * (target_q - agent.alpha * log_probs)

    # Calculate MSELoss
    Q1, Q2 = agent.Q(obs_batch, act_batch)
    Q_loss1 = torch.mean((Q1 - TQ)**2)
    Q_loss2 = torch.mean((Q2 - TQ)**2)
    Q_loss = Q_loss1 + Q_loss2

    # Gradient descent
    agent.Q_optimizer.zero_grad()
    Q_loss.backward()
    agent.Q_optimizer.step()
```

trick! (why?)

$\longleftarrow$ training critics

Learning and Decision Systems

SAC - Implementation

```
1       actions, log_probs = agent.pi(obs_batch, with_log_prob=True)
2
3       freeze(agent.Q)
4       q1, q2 = agent.Q(obs_batch, actions)
5       q = torch.min(q1, q2)
6
7       # Need to perform gradient ascent, so (-) is required
8       # TODO: build policy loss
9       #pi_loss = torch.mean( #TODO )
10      pi_loss = torch.mean(agent.alpha * log_probs - q)
11
12      # Gradient ascent
13      agent.pi_optimizer.zero_grad()
14      pi_loss.backward()
15      agent.pi_optimizer.step()
```

$\longleftarrow$ training actor

SAC - Automatic Adjustment of temperature parameter

Unconstrained optimization

$$\sum_{t=0}^{T-1} \left(r(s_t, a_t) + \alpha \mathscr{H}(\pi(\cdot|s_t)) \right)$$

Constrained optimization

$$\max_{\pi_0} \mathbb{E}_{\rho_\pi} \left[\sum_{t=0}^{T} r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

$$\text{s.t.} \quad \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} [-\log(\pi_t(\mathbf{a}_t \mid \mathbf{s}_t))] \geq \mathcal{H} \quad \forall t$$

maximize rewards while ensuring entropy remains above $\mathcal{H}$

SAC - Automatic Adjustment of temperature parameter

approximation (DP, duality, ..)

$$\max_{\pi_0} \mathbb{E}_{\rho_\pi}\left[\sum_{t=0}^{T} r(\mathbf{s}_t, \mathbf{a}_t)\right]$$
$$\text{s.t.} \quad \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim\rho_\pi}[-\log(\pi_t(\mathbf{a}_t \mid \mathbf{s}_t))] \geq \mathcal{H} \quad \forall t$$

$$J(\alpha) = \alpha \mathbb{E}_{a_t\sim\pi_t}[H(\pi(\cdot|s_t)) - \bar{\mathcal{H}}]$$

Learning and Decision Systems

SAC - Automatic Adjustment of temperature parameter

approximation (DP, duality, ..)

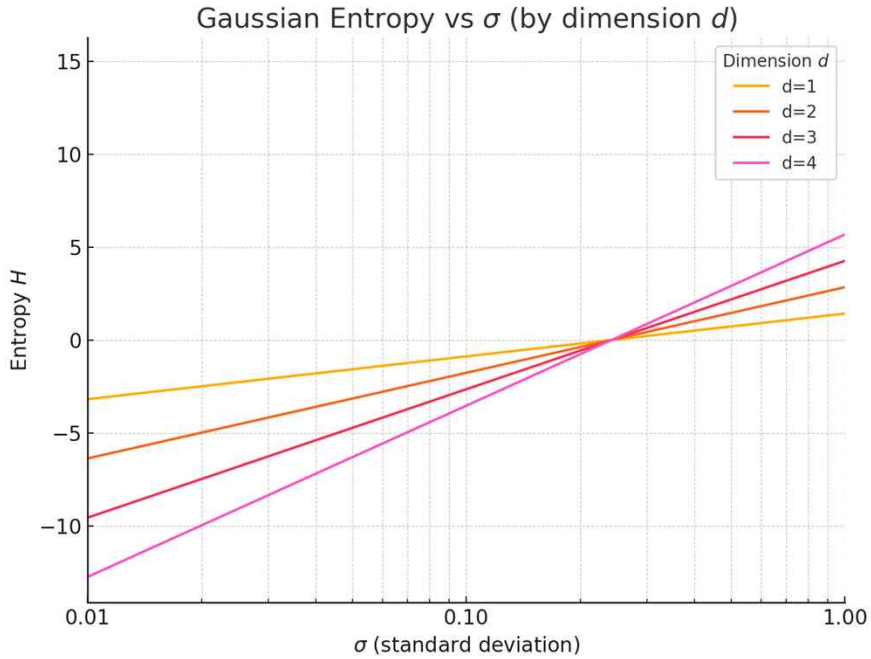
$$\max_{\pi_0} \mathbb{E}_{\rho_\pi}\left[\sum_{t=0}^{T} r(\mathbf{s}_t, \mathbf{a}_t)\right]$$
$$\text{s.t.} \quad \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim\rho_\pi}[-\log(\pi_t(\mathbf{a}_t \mid \mathbf{s}_t))] \geq \mathcal{H} \quad \forall t$$

$$J(\alpha) = \alpha \mathbb{E}_{a_t\sim\pi_t}[H(\pi(\cdot|s_t)) - \bar{\mathcal{H}}]$$

In practice, $\bar{\mathcal{H}}$ = -dim($\mathcal{A}$), why?

Learning and Decision Systems

SAC - Automatic Adjustment of temperature parameter



$$p_d := \mathcal{N}(0, \sigma I_d)$$

$$H(p_d) = \frac{d}{2} \underbrace{\log(2\pi e \sigma^2)}_{\text{negative if } \sigma \lessapprox 0.24....}$$

Learning and Decision Systems

Thank you